

A PROJECT REPORT ON

FINANCIAL MANAGEMENT SYSTEM USING SQL

Submitted By:
Chanduri Jaswanth

Table of Contents

S.No	Content	Page no
1	Introduction	1
2	Project Objective	1
3	Technologies Used	1
4	Database Design	1
5	Tables Used in Project	2 - 3
6	Module 1: Database Creation	4 - 5
7	Module 2: Data Modelling	6 - 7
8	Module 3: Raw Data Insertion	8 - 9
9	Module 4: Data Cleaning	10
10	Module 5: Data Manipulation	11
11	Module 6: SQL Queries & Reports	12 - 14
12	Module 7: TCL Commands	15
13	Module 8: DCL Commands	16
14	Module 9: Alter & Drop Operations	17
15	Conclusion	18

Introduction

The Financial Management System is a database management project developed using Oracle SQL. The project is designed to manage customer accounts, banking transactions, loans, and employee information. It demonstrates the implementation of SQL concepts such as DDL, DML, TCL, DCL, Data Cleaning, Data Modeling, Joins, Constraints, and Aggregate Functions.

Project Objective

The objective of this project is:

- To understand relational database concepts
- To implement SQL commands practically
- To manage banking-related information efficiently
- To perform data cleaning and manipulation
- To generate reports using SQL queries
- To understand transaction and permission management

Technologies Used

- Oracle SQL - Database Management System
- DDL - Data Definition Language
- DML - Data Manipulation Language
- DCL - Data Control Language
- TCL - Transaction Control Language

Database Design

The project contains the following tables:

- Customers
- Accounts
- Transactions
- Loans
- Employees

Relationships:

- One customer can have multiple accounts
- One account can contain multiple transactions
- Loans are connected to customers

Tables Used in Project

Customer Table

Column Name	Data Type	Constraint
customer_ID	NUMBER(10)	PRIMARY KEY
customer_name	VARCHAR2(50)	NOT NULL
email	VARCHAR2(100)	UNIQUE
phn_number	NUMBER(10)	-
city	VARCHAR2(50)	-

Accounts Table

Column Name	Data Type	Constraint
account_ID	NUMBER(12)	PRIMARY KEY
customer_ID	NUMBER(10)	FOREIGN KEY
account_type	VARCHAR2(20)	CHECK
balance	NUMBER(10,2)	-
open_date	DATE	-

Transactions Table

Column Name	Data Type	Constraint
transaction_ID	NUMBER(10)	PRIMARY KEY
account_ID	NUMBER(12)	FOREIGN KEY
transaction_type	VARCHAR2(20)	CHECK
amount	NUMBER(10,2)	-
transaction_date	DATE	-

Loans Table

Column Name	Data Type	Constraint
loan_ID	NUMBER(10)	PRIMARY KEY
customer_ID	NUMBER(10)	FOREIGN KEY
loan_type	VARCHAR2(50)	-
loan_amount	NUMBER(10,2)	-
loan_status	VARCHAR2(20)	CHECK

Employees Table

Column Name	Data Type	Constraint
employee_ID	NUMBER(10)	PRIMARY KEY
employee_name	VARCHAR2(20)	NOT NULL
designation	VARCHAR2(20)	NOT NULL
salary	NUMBER(10,2)	-

Module 1: Database Creation

In this module, we created the database and designed tables required for the Financial Management System. We created tables like Customers, Accounts, Transactions, Loans, and Employees with proper constraints and keys.

Customers Table

```
CREATE TABLE customers(customer_ID NUMBER(10) PRIMARY KEY,
customer_name VARCHAR2(50) NOT NULL, email VARCHAR2(100) UNIQUE,
phn_number NUMBER(10), city VARCHAR2(50));
```

Name	Null?	Type
CUSTOMER_ID	NOT NULL	NUMBER(10)
CUSTOMER_NAME	NOT NULL	VARCHAR2(50)
EMAIL		VARCHAR2(100)
PHONE_NUMBER		NUMBER(10)
CITY		VARCHAR2(50)
PAN_NUMBER		VARCHAR2(20)

Accounts Table

```
CREATE TABLE accounts(account_ID NUMBER(12) PRIMARY KEY, customer_ID
NUMBER(10) REFERENCES customers(customer_ID), account_type VARCHAR2(20)
CHECK(account_type IN('savings','current')), balance NUMBER(10,2), open_date
DATE);
```

Name	Null?	Type
ACCOUNT_ID	NOT NULL	NUMBER(12)
CUSTOMER_ID		NUMBER(10)
ACCOUNT_TYPE		VARCHAR2(20)
BALANCE		NUMBER(10,2)
OPEN_DATE		DATE

Transactions Table

```
CREATE TABLE transactions(transaction_id NUMBER(10) PRIMARY KEY,
account_ID NUMBER(12) REFERENCES accounts(account_ID), transaction_type
VARCHAR2(20) CHECK(transaction_type IN('Credit','Debit')), amount NUMBER(10,2),
transaction_date DATE);
```

Name	Null?	Type
TRANSACTION_ID	NOT NULL	NUMBER(10)
ACCOUNT_ID		NUMBER(12)
TRANSACTION_TYPE		VARCHAR2(20)
AMOUNT		NUMBER(10,2)
TRANSACTION_DATE		DATE

Loans Table

```
CREATE TABLE loans(loan_ID NUMBER(10) PRIMARY KEY, customer_ID
NUMBER(10) REFERENCES customers(customer_ID), loan_type VARCHAR2(50),
loan_amount NUMBER(10,2), loan_status VARCHAR2(20) CHECK(loan_status
IN('Approved','Rejected','Pending')));
```

Name	Null?	Type
LOAN_ID	NOT NULL	NUMBER(10)
CUSTOMER_ID		NUMBER(10)
LOAN_TYPE		VARCHAR2(50)
LOAN_AMOUNT		NUMBER(10,2)
LOAN_STATUS		VARCHAR2(20)

Employees Table

```
CREATE TABLE employees(employee_ID NUMBER(10) PRIMARY KEY, employee_name
VARCHAR2(20) NOT NULL, designation VARCHAR2(20) NOT NULL, salary NUMBER(10,2));
```

Name	Null?	Type
EMPLOYEE_ID	NOT NULL	NUMBER(10)
EMPLOYEE_NAME	NOT NULL	VARCHAR2(20)
DESIGNATION	NOT NULL	VARCHAR2(20)
SALARY		NUMBER(10,2)

Module 2: Data Modelling

1. One customer can have multiple accounts. How will you design the relationship?

This is a **one-to-many relationship**.

One customer can have many accounts, but each account belongs to one customer.

So, customer_ID should be a foreign key in the accounts table.

```
SQL> create table customers(customer_ID number(10) PRIMARY KEY, customer_name varchar2(50) NOT NULL, email varchar2(100) UNIQUE, phn_number number(10), city varchar2(50));  
  
Table created.  
  
SQL> create table accounts(account_ID number(12) PRIMARY KEY, customer_ID number(10) REFERENCES customers(customer_ID), account_type varchar2(20) CHECK(account_type IN('savings', 'current')), balance number(10, 2), open_date DATE);  
  
Table created.
```

2. How will you connect transactions with accounts?

This is also a **one-to-many relationship**.

One account can have multiple transactions.

So, account_ID in the transactions table should be a foreign key referencing the accounts table.

```
SQL> create table transactions(transaction_id number(10) PRIMARY KEY, account_ID number(12) REFERENCES accounts(account_ID), transaction_type varchar2(20) CHECK(transaction_type IN('Credit', 'Debit')), amount number(10, 2), transaction_date DATE);  
  
Table created.
```

3. Which columns should be Primary Keys?

Primary Keys:

- customer_ID → customers table
- account_ID → accounts table

- transaction_ID → transactions table
- loan_ID → loans table
- employee_id → Employees table

4. Which columns should not allow NULL values?

Columns that contain essential information should use NOT NULL.

- customer_ID
- customer_name
- account_ID
- Transaction_ID
- loan_ID
- employee_ID
- employee_name
- designation

Module 3: Raw Data Insertion

In this module, we inserted customer records, account details, transaction records, employee details, and loan information. We also inserted duplicate and NULL values for practicing data cleaning operations.

Insert minimum:

- 10 Customers
- 10 Accounts
- 20 Transactions
- 5 Employees

Requirements:

- Include duplicate values
- Include NULL values
- Include invalid balances
- Include invalid transaction amounts

Insertion of Rows in Customers Table:

> **INSERT INTO customers VALUES (101, 'Ram', 'ram@gmail.com', 9876543210, 'Hyderabad');**

```
SQL> INSERT INTO customers
  2 VALUES (101, 'Ram', 'ram@gmail.com', 9876543210, 'Hyderabad');

1 row created.

SQL>
SQL> INSERT INTO customers
  2 VALUES (102, 'Sita', 'sita@gmail.com', 9123456780, 'Chennai');

1 row created.

SQL>
SQL> INSERT INTO customers
  2 VALUES (103, 'John', 'john@gmail.com', 9988776655, 'Bangalore');

1 row created.

SQL>
SQL> INSERT INTO customers
  2 VALUES (104, 'Priya', 'priya@gmail.com', 9871234567, 'Mumbai');

1 row created.

SQL>
SQL> INSERT INTO customers
  2 VALUES (105, 'Amit', 'amit@gmail.com', 9012345678, 'Delhi');

1 row created.

SQL>
SQL> INSERT INTO customers
  2 VALUES (106, 'Kiran', NULL, 9876543201, 'Hyderabad');

1 row created.

SQL>
SQL> INSERT INTO customers
  2 VALUES (107, 'Ravi', 'ravi@gmail.com', NULL, 'Pune');

1 row created.
```

Insertion of Rows in Accounts Table:

> INSERT INTO accounts VALUES (1001, 101, 'savings', 5000, TO_DATE('01-01-2025','DD-MM-YYYY'));

```
SQL> INSERT INTO accounts
  2 VALUES (1001, 101, 'savings', 5000,
  3 TO_DATE('01-01-2025','DD-MM-YYYY'));
1 row created.

SQL>
SQL> INSERT INTO accounts
  2 VALUES (1002, 102, 'current', 12000,
  3 TO_DATE('02-01-2025','DD-MM-YYYY'));
1 row created.

SQL>
SQL> INSERT INTO accounts
  2 VALUES (1003, 103, 'savings', 7000,
  3 TO_DATE('03-01-2025','DD-MM-YYYY'));
1 row created.

SQL>
SQL> INSERT INTO accounts
  2 VALUES (1004, 104, 'current', 15000,
  3 TO_DATE('04-01-2025','DD-MM-YYYY'));
1 row created.

SQL>
SQL> INSERT INTO accounts
  2 VALUES (1005, 105, 'savings', 9000,
  3 TO_DATE('05-01-2025','DD-MM-YYYY'));
1 row created.

SQL>
SQL> INSERT INTO accounts
  2 VALUES (1006, 106, 'current', NULL,
  3 TO_DATE('06-01-2025','DD-MM-YYYY'));
1 row created.

SQL>
SQL> INSERT INTO accounts
  2 VALUES (1007, 107, 'savings', -500,
  3 TO_DATE('07-01-2025','DD-MM-YYYY'));
1 row created.
```

Insertion of Rows in Transactions Table:

> INSERT INTO transactions VALUES (1, 1001, 'Credit', 2000, TO_DATE('01-03-2025','DD-MM-YYYY'));

```
SQL> INSERT INTO transactions VALUES (1, 1001, 'Credit', 2000, TO_DATE('01-03-2025','DD-MM-YYYY'));
1 row created.

SQL>
SQL> INSERT INTO transactions VALUES (2, 1001, 'Debit', 1000, TO_DATE('02-03-2025','DD-MM-YYYY'));
1 row created.

SQL>
SQL> INSERT INTO transactions VALUES (3, 1002, 'Credit', 5000, TO_DATE('03-03-2025','DD-MM-YYYY'));
1 row created.

SQL>
SQL> INSERT INTO transactions VALUES (4, 1002, 'Debit', 2000, TO_DATE('04-03-2025','DD-MM-YYYY'));
1 row created.

SQL>
SQL> INSERT INTO transactions VALUES (5, 1003, 'Credit', 3000, TO_DATE('05-03-2025','DD-MM-YYYY'));
1 row created.

SQL>
SQL> INSERT INTO transactions VALUES (6, 1003, 'Debit', 1000, TO_DATE('06-03-2025','DD-MM-YYYY'));
1 row created.

SQL>
SQL> INSERT INTO transactions VALUES (7, 1004, 'Credit', 4000, TO_DATE('07-03-2025','DD-MM-YYYY'));
1 row created.

SQL>
SQL> INSERT INTO transactions VALUES (8, 1004, 'Debit', 1500, TO_DATE('08-03-2025','DD-MM-YYYY'));
1 row created.

SQL>
SQL> INSERT INTO transactions VALUES (9, 1005, 'Credit', 6000, TO_DATE('09-03-2025','DD-MM-YYYY'));
1 row created.

SQL>
SQL> INSERT INTO transactions VALUES (10, 1005, 'Debit', 2500, TO_DATE('10-03-2025','DD-MM-YYYY'));
```

Module 4: Data Cleaning

In this module, we identified NULL values, duplicate records, negative balances, and invalid transactions.

We used UPDATE and DELETE commands to clean the data.

1. Find NULL Email Addresses

> `SELECT * FROM customers WHERE email IS NULL;`

2. Replacing Missing Phone Number

> `UPDATE customers SET phn_number = 9999999999 WHERE phn_number IS NULL;`

3. Remove accounts having negative balances.

> `DELETE FROM accounts WHERE balance < 0;`

4. Delete invalid transactions.

> `DELETE FROM transactions WHERE amount < 0;`

5. Identify duplicate customer records.

> `SELECT customer_name, COUNT(*) FROM customers GROUP BY customer_name HAVING COUNT(*) > 1;`

6. Standardize city names into uppercase.

> `UPDATE customers SET city = UPPER(city);`

```
SQL> SELECT * FROM customers WHERE email IS NULL;

CUSTOMER_ID CUSTOMER_NAME  EMAIL                                PHN_NUMBER CITY
-----
106 Kiran                                9876543201 Hyderabad

SQL> UPDATE customers SET phn_number = 9999999999 WHERE phn_number IS NULL;

1 row updated.

SQL> DELETE FROM accounts WHERE balance < 0;
DELETE FROM accounts WHERE balance < 0
*
ERROR at line 1:
ORA-02292: integrity constraint (SYSTEM.SYS_C004104) violated - child record found

SQL> DELETE FROM transactions WHERE amount < 0;

3 rows deleted.

SQL> SELECT customer_name, COUNT(*) FROM customers GROUP BY customer_name HAVING COUNT(*) > 1;

CUSTOMER_NAME  COUNT(*)
-----
Ram                2

SQL> UPDATE customers SET city = UPPER(city);

12 rows updated.
```

Module 5: Data Manipulation

In this module, we updated balances, modified customer information, added new customers, and deleted inactive accounts. This module helped us understand real-time data management.

1. Update account balances after transactions.

> UPDATE accounts SET balance = balance + 5000 WHERE account_ID = 1001;

> UPDATE accounts SET balance = balance - 2000 WHERE account_ID = 1002;

2. Add new customers into the system.

> INSERT INTO customers VALUES (113, 'Kavya', 'kavya@gmail.com', 9876501234, 'Hyderabad');

3. Delete inactive customer accounts.

> DELETE FROM accounts WHERE balance = 0;

4. Modify customer contact details.

> UPDATE customers SET phn_number = 9999988888 WHERE customer_ID = 101;

> UPDATE customers SET email = 'ram_new@gmail.com' WHERE customer_ID = 101;

5. Increase loan amount for selected customers.

> UPDATE loans SET loan_amount = loan_amount + 10000 WHERE customer_ID = 101;

```
SQL> UPDATE accounts SET balance = balance + 5000 WHERE account_ID = 1001;
1 row updated.

SQL> UPDATE accounts SET balance = balance - 2000 WHERE account_ID = 1002;
1 row updated.

SQL> INSERT INTO customers VALUES (113, 'Kavya', 'kavya@gmail.com', 9876501234, 'Hyderabad');
1 row created.

SQL> DELETE FROM accounts WHERE balance = 0;
DELETE FROM accounts WHERE balance = 0
*
ERROR at line 1:
ORA-02292: integrity constraint (SYSTEM.SYS_C004104) violated - child record found

SQL> UPDATE customers SET phn_number = 9999988888 WHERE customer_ID = 101;
1 row updated.

SQL> UPDATE customers SET email = 'ram_new@gmail.com' WHERE customer_ID = 101;
1 row updated.

SQL> UPDATE loans SET loan_amount = loan_amount + 10000 WHERE customer_ID = 101;
0 rows updated.
```

Module 6: SQL Queries & Reports

In this module, we used SELECT queries, JOIN queries, and aggregate functions. We generated reports such as total balance, average balance, and transaction history.

Basic Queries:

1. Display all customers.

> SELECT * FROM customers;

2. Show all account details.

> SELECT * FROM accounts;

3. Find customers from Hyderabad.

> SELECT * FROM customers WHERE city = 'HYDERABAD';

4. Display accounts with balance greater than 50,000.

> SELECT * FROM accounts WHERE balance > 50000;

```
SQL> SELECT * FROM customers;
```

CUSTOMER_ID	CUSTOMER_NAME	EMAIL	PHN_NUMBER	CITY
101	Ram	ram_new@gmail.com	9999988888	HYDERABAD
102	Sita	sita@gmail.com	9123456780	CHENNAI
103	John	john@gmail.com	9988776655	BANGALORE
104	Priya	priya@gmail.com	9871234567	MUMBAI
105	Amit	amit@gmail.com	9012345678	DELHI
106	Kiran		9876543201	HYDERABAD
107	Ravi	ravi@gmail.com	9999999999	PUNE
108	Neha	neha@gmail.com	9876543203	CHENNAI
109	Arun	arun@gmail.com	9876543204	KOLKATA
110	Divya	divya@gmail.com	9876543205	HYDERABAD
111	Ram	ram2@gmail.com	9876500000	MUMBAI

13 rows selected.

```
SQL> SELECT * FROM accounts;
```

ACCOUNT_ID	CUSTOMER_ID	ACCOUNT_TYPE	BALANCE	OPEN_DATE
1002	102	current	10000	02-JAN-25
1003	103	savings	7000	03-JAN-25
1004	104	current	15000	04-JAN-25
1005	105	savings	9000	05-JAN-25
1006	106	current	-500	06-JAN-25
1007	107	savings	15000	07-JAN-25
1008	108	current	5000	08-JAN-25
1009	109	savings	0	09-JAN-25
1010	110	current	-5000	10-JAN-25
1011	101	savings	15000	15-JAN-25
1001	101	savings	15000	01-JAN-25

11 rows selected.

```
SQL> SELECT * FROM customers WHERE city = 'HYDERABAD';
```

CUSTOMER_ID	CUSTOMER_NAME	EMAIL	PHN_NUMBER	CITY
101	Ram	ram_new@gmail.com	9999988888	HYDERABAD
106	Kiran		9876543201	HYDERABAD
110	Divya	divya@gmail.com	9876543205	HYDERABAD
112	Anu	anu@gmail.com	9876543000	HYDERABAD

JOIN Queries:

5. Show customer name with account details.

> `SELECT c.customer_name, a.account_ID, a.account_type, a.balance FROM customers c JOIN accounts a ON c.customer_ID = a.customer_ID;`

6. Display transaction history for each customer.

> `SELECT c.customer_name, t.transaction_id, t.transaction_type, t.amount FROM customers c JOIN accounts a ON c.customer_ID = a.customer_ID JOIN transactions t ON a.account_ID = t.account_ID;`

7. Find customers who made transactions above 10,000.

> `SELECT DISTINCT c.customer_name FROM customers c JOIN accounts a ON c.customer_ID = a.customer_ID JOIN transactions t ON a.account_ID = t.account_ID WHERE t.amount > 10000;`

8. Show account type-wise total balance.

> `SELECT account_type, SUM(balance) AS total_balance FROM accounts GROUP BY account_type;`

```
SQL> SELECT c.customer_name, a.account_ID, a.account_type, a.balance FROM customers c JOIN accounts a ON c.customer_ID = a.customer_ID;
```

CUSTOMER_NAME	ACCOUNT_ID	ACCOUNT_TYPE	BALANCE
Sita	1002	current	10000
John	1003	savings	7000
Priya	1004	current	15000
Amit	1005	savings	9000
Kiran	1006	current	
Ravi	1007	savings	-500
Meha	1008	current	15000
Arun	1009	savings	5000
Divya	1010	current	0
Ram	1011	savings	-5000
Ram	1001	savings	15000

11 rows selected.

```
SQL> SELECT c.customer_name, t.transaction_id, t.transaction_type, t.amount FROM customers c JOIN accounts a ON c.customer_ID = a.customer_ID JOIN transactions t ON a.account_ID = t.account_ID;
```

CUSTOMER_NAME	TRANSACTION_ID	TRANSACTION_TYPE	AMOUNT
Ram	1	Credit	2000
Ram	2	Debit	1000
Sita	3	Credit	5000
Sita	4	Debit	2000
John	5	Credit	3000
John	6	Debit	1000
Priya	7	Credit	4000
Priya	8	Debit	1500
Amit	9	Credit	6000
Amit	10	Debit	2500
Kiran	11	Credit	

CUSTOMER_NAME	TRANSACTION_ID	TRANSACTION_TYPE	AMOUNT
Ravi	13	Credit	3500
Ravi	14	Debit	1200
Meha	15	Credit	7000
Meha	16	Debit	3000
Arun	17	Credit	4500
Arun	18	Debit	2000
Divya	19	Credit	5000

18 rows selected.

Aggregate Functions:

9. Find total bank balance.

> `SELECT SUM(balance) AS total_bank_balance FROM accounts;`

10. Find average account balance.

> `SELECT AVG(balance) AS average_balance FROM accounts;`

11. Find maximum transaction amount.

> `SELECT MAX(amount) AS maximum_transaction_amount FROM transactions;`

12. Count total customers.

> `SELECT COUNT(*) AS total_customers FROM customers;`

```
SQL> SELECT SUM(balance) AS total_bank_balance FROM accounts;

TOTAL_BANK_BALANCE
-----
              70500

SQL> SELECT AVG(balance) AS average_balance FROM accounts;

AVERAGE_BALANCE
-----
              7050

SQL> SELECT MAX(amount) AS maximum_transaction_amount FROM transactions;

MAXIMUM_TRANSACTION_AMOUNT
-----
                   7000

SQL> SELECT COUNT(*) AS total_customers FROM customers;

TOTAL_CUSTOMERS
-----
              13
```

Module 7: TCL Commands

In this module, we learned transaction management using COMMIT, ROLLBACK, and SAVEPOINT commands. These commands are important in banking systems for secure transactions.

1. Start a transaction for money transfer.

> SAVEPOINT before_transfer;

2. Deduct amount from one account.

> UPDATE accounts SET balance = balance - 5000 WHERE account_ID = 1001;

3. Add amount into another account.

> UPDATE accounts SET balance = balance + 5000 WHERE account_ID = 1002;

4. Create SAVEPOINT.

> SAVEPOINT transfer_done;

5. Rollback the transaction.

> ROLLBACK TO before_transfer;

6. Commit successful transaction.

> COMMIT;

```
SQL> SAVEPOINT before_transfer;
Savepoint created.
SQL> UPDATE accounts SET balance = balance - 5000 WHERE account_ID = 1001;
1 row updated.
SQL> UPDATE accounts SET balance = balance + 5000 WHERE account_ID = 1002;
1 row updated.
SQL> SAVEPOINT transfer_done;
Savepoint created.
SQL> ROLLBACK TO before_transfer;
Rollback complete.
SQL> COMMIT;
Commit complete.
```

Module 8: DCL Commands

In this module, we learned about database security using GRANT and REVOKE commands.

These commands are used to control access permissions for users.

1. Create a new database user.

> `CREATE USER finance_user IDENTIFIED BY finance123;`

2. Grant SELECT and INSERT permissions.

> `GRANT SELECT, INSERT ON customers TO finance_user;`

> `GRANT SELECT, INSERT ON accounts TO finance_user;`

3. Revoke DELETE permissions.

> `REVOKE DELETE ON customers FROM finance_user;`

4. Provide limited access to finance employees.

> `CREATE USER employee_user IDENTIFIED BY emp123;`

> `GRANT SELECT ON accounts TO employee_user;`

> `GRANT SELECT ON transactions TO employee_user;`

```
SQL> CREATE USER finance_user IDENTIFIED BY finance123;
User created.

SQL> GRANT SELECT, INSERT ON customers TO finance_user;
Grant succeeded.

SQL> GRANT SELECT, INSERT ON accounts TO finance_user;
Grant succeeded.

SQL> REVOKE DELETE ON customers FROM finance_user;
REVOKE DELETE ON customers FROM finance_user
*
ERROR at line 1:
ORA-01927: cannot REVOKE privileges you did not grant

SQL> CREATE USER employee_user IDENTIFIED BY emp123;
User created.

SQL> GRANT SELECT ON accounts TO employee_user;
Grant succeeded.

SQL> GRANT SELECT ON transactions TO employee_user;
Grant succeeded.
```

Module 9: Alter & Table Operations

In this module, we used ALTER TABLE commands to add and rename columns and DROP commands to remove unnecessary tables.

1. Add PAN Number column to Customers table.

> ALTER TABLE customers ADD pan_number VARCHAR2(20);

2. Add Loan Status column.

> ALTER TABLE loans ADD loan_status VARCHAR2(20);

3. Rename a column.

> ALTER TABLE customers RENAME COLUMN phn_number TO phone_number;

4. Drop unnecessary table.

> DROP TABLE employees;

```
SQL> ALTER TABLE customers ADD pan_number VARCHAR2(20);
Table altered.

SQL> ALTER TABLE loans ADD loan_status VARCHAR2(20);
ALTER TABLE loans ADD loan_status VARCHAR2(20)
*
ERROR at line 1:
ORA-01430: column being added already exists in table

SQL> ALTER TABLE customers RENAME COLUMN phn_number TO phone_number;
Table altered.

SQL> DROP TABLE employees;
Table dropped.
```

Conclusion

The Financial Management System project provided practical knowledge of database management using Oracle SQL. Through this project, various SQL concepts such as DDL, DML, TCL, DCL, Joins, Constraints, Data Cleaning, and Data Manipulation were implemented successfully. The project helped in understanding how real-world banking systems manage customers, accounts, transactions, loans, and employee information efficiently.

By creating tables, inserting and cleaning data, generating reports, and managing transactions and permissions, the project improved problem-solving skills and practical SQL knowledge. Overall, this project serves as a strong foundation for understanding relational database systems and real-time financial data management applications.